Deploying the meal-nutrition API

Upload one meal photo and get back every dish with calories, protein, carbohydrate, fat and fibre

Package **food-plate-api.zip** (366 MB) · API 1.0.0 · model version `76ec58bc09aa` · 110 dishes recognised · 27 October 2026

At a glance

- **Server:** any Linux machine with Docker. 4 vCPU and 4–8 GB RAM is enough for testing (about 1.5 s per photo). **No GPU needed.**
- **Deploy in 3 commands**: unzip (or *git clone*), *docker build*, *docker run* (section 3). Then run the included smoke test (section 4).
- **Self-contained:** both models and the nutrition table are inside the zip; nothing is downloaded at run time and no photo is ever stored.
- **Accuracy** on 136 real meal photos never used in training: 84% of dishes found, 75% of those named right (90% in the top 3), meal calories within a median 8%. Portions are standard servings, not measured from the photo.

1. What is in the package

Path	What it is
`Deployment_Guide.pdf`	this guide
`DEPLOYMENT.md`	the same guide as a text file
`Dockerfile`	builds the server image (Python 3.10, CPU-only PyTorch)
`requirements-serve.txt`	exact package versions the API was tested with
`plate/`	API code (plate/api.py) and the analysis pipeline
`models/yolo/detector.pt`	dish detector: finds each bowl, plate or piece (YOLO11m, fine-tuned, 39 MB)
`models/plate_classifier/`	dish classifier: names each box, 110 dishes (SigLIP-B/16, fine-tuned, 355 MB)
`data/nutrition/dish_nutrition.csv`	nutrition per 100 g and standard serving for 113 dishes, with the source of every value
`api_smoke_test.py`	end-to-end check against a running server (needs only Python)
`samples/`	4 real meal photos for the smoke test
`VERSION.json`	model version
`SHA256SUMS.txt`	checksums: *sha256sum -c SHA256SUMS.txt* confirms nothing was corrupted

2. The server you need

Item	Recommendation
CPU	4 vCPU for testing; more vCPUs make each photo faster (table below)
RAM	about 1.2 GB per worker process; 4 GB for one worker, 8 GB to be comfortable
Disk	about 5 GB free (Docker image 2–3 GB plus the build cache)
OS	any Linux with Docker (Ubuntu 22.04/24.04 is fine); Windows or macOS also work
GPU	not needed

Item	Recommendation
Network	inbound port 443 (HTTPS) through a reverse proxy; the API itself listens on 8000

Time per photo with the models loaded, by CPU threads (measured on the development laptop; cloud vCPUs are usually a little slower):

CPU threads	1	2	4	8
Median time per photo	5.5 s	2.8 s	1.5 s	1.0 s

3. Deploy

Option A: Docker (recommended)

Step 1. Get the files onto the server. Either copy **food-plate-api.zip** there and unzip it, or clone the private GitHub repository (ask for access). The model files are stored with Git LFS, so install git-lfs before cloning:

```
unzip food-plate-api.zip && cd food-plate-api          # either this ...

git lfs install                                        # ... or this
git clone https://github.com/dhruv-upmanyu/food-plate-api.git && cd food-plate-api

sha256sum -c SHA256SUMS.txt --quiet                    # optional: no output = every file is intact
```

Then build the image (several minutes the first time; it downloads the Python packages):

```
docker build -t food-plate-api .
```

Step 2. Start it, with a secret key of your choice. It restarts automatically after a reboot:

```
docker run -d --name food-api --restart unless-stopped -p 8000:8000 \
  -e PLATE_API_KEY=<your-secret-key> food-plate-api
```

Step 3. Wait 10–20 s for the models to load. Then **http://<server>:8000/health** returns *"status": "ok"* and interactive API docs are at **http://<server>:8000/docs**. Logs: `docker logs -f food-api`. Stop: `docker stop food-api`.

> **The Docker build hasn't been run yet.** Docker wasn't available on the development laptop. The same code, models and pinned package versions were tested by running the API directly (option B), and every check passed. Please treat the first *docker build* as the first test of the Dockerfile, and send back the error output if it fails.

Option B: without Docker (Python 3.10)

```
python3.10 -m venv venv && . venv/bin/activate         # Windows: venv\Scripts\activate
pip install --index-url https://download.pytorch.org/whl/cpu torch==2.11.0 torchvision==0.26.0
pip install -r requirements-serve.txt
sudo apt install libgl1 libglib2.0-0                   # bare Linux only: libraries OpenCV needs
PLATE_API_KEY=<your-secret-key> uvicorn plate.api:app --host 0.0.0.0 --port 8000
```

To keep it running after logout, run the last line as a systemd service, or use Docker.

4. Check that it works

From any machine with Python (no packages needed):

```
python api_smoke_test.py --url http://<server>:8000 --key <your-secret-key>
```

It checks /health, /v1/dishes and the error handling, then analyses the 4 sample photos and prints each dish with its nutrition. The last line must be **ALL CHECKS PASSED**. Example output for one photo:

```
thali_chole_roti.jpg: 4 dishes, server 975 ms
  dish                         count  grams   kcal  prot  carb   fat fibre  conf
  Chole / chana masala             1    150    245   9.2  30.0  10.3   7.1  0.71
  Roti / chapati                   1     40     81   2.4  14.3   1.4   2.5  1.00
  Curd (dahi)                      1    100     61   3.5   4.7   3.2   0.0  0.98
  Salad (cucumber, tomato, oni     1    100     30   1.0   5.5   0.3   2.6  1.00
  MEAL TOTAL                            390    417  16.1  54.5  15.2  12.2
```

The samples deliberately include typical mistakes, so they are not a surprise during testing:

Sample	Labelled dishes	API result
`thali_chole_roti.jpg`	chole, roti, curd, salad	all 4 right
`idli_vada_sambar.jpg`	2 idli, vada, 2 chutneys	idli ×2 and vada right; finds 1 of the 2 chutneys
`thali_roti_rice_6_dishes.jpg`	rice, roti, mixed veg, dry sabzi, curd, salad	5 of 6 right; the dry sabzi is named egg bhurji
`rajma_rice_bhindi.jpg`	rajma, rice, bhindi, curd, salad	rice, bhindi, curd, salad right; the rajma poured over the rice is missed and that region counts as plain rice

5. Put it on the internet safely

- **HTTPS:** put a reverse proxy or the cloud load balancer in front of port 8000. With Caddy (certificates are automatic), the whole Caddyfile is the 3 lines below. With nginx, set `client_max_body_size 16m;` and `proxy_read_timeout 60s;`.
- **API key:** always set PLATE_API_KEY on a public server. Call the API from your app's backend, so the key is never shipped inside a mobile app. Add rate limiting at the proxy.
- **Firewall:** open only 443 (and 80 for the certificate), not 8000.
- **Browser apps:** if a website calls the API directly, list its origin in PLATE_CORS_ORIGINS. Mobile apps and server-to-server calls don't need this.
- **Privacy:** photos are processed in memory and never written to disk or logged; the access log records only the path, status and client IP. If you later want to keep photos to improve the model, get users' consent first.
- **No phone-home:** the API disables Ultralytics' usage analytics and run-time package installs.

```
food.example.com {
    reverse_proxy 127.0.0.1:8000
}
```

6. Settings (environment variables, all optional)

Variable	Default	Purpose
PLATE_API_KEY	none (open)	requires the X-API-Key header on /v1/*; **set it on any public server**
PLATE_CORS_ORIGINS	none	comma-separated website origins allowed to call the API from a browser
PLATE_MAX_UPLOAD_MB	15	largest photo accepted
PLATE_TORCH_THREADS	all CPUs	CPU threads per worker; set it to the container's --cpus value, or to CPUs ÷ workers
PLATE_DET_CONF	0.25	detector threshold (chosen on validation photos); lower values find more dishes and more false ones

Example: `docker run ... --cpus 4 -e PLATE_TORCH_THREADS=4 -e PLATE_API_KEY=... food-plate-api`

7. Scaling

- One worker analyses one photo at a time; other requests wait in a queue. Throughput is about 1 ÷ (time per photo) per worker.
- For more traffic, run more containers behind a load balancer (simplest), or more workers on a bigger machine: `uvicorn plate.api:app --workers 2 ...` with PLATE_TORCH_THREADS = vCPUs ÷ 2 and about 1.2 GB RAM per worker.
- Serverless containers (Cloud Run, App Runner, Azure Container Apps and similar) work, but a new instance spends 10–20 s loading the models. Keep at least one instance always running.
- /health reports `cpu_threads` and `model_version`: use it to check the settings took effect and which models are live.

8. Using the API (for the app developers)

Endpoint	What it does
POST /v1/analyze	multipart upload, field **image** (JPEG, PNG or WebP; up to 15 MB and 50 megapixels): every dish found, plus meal totals
GET /v1/dishes	all 113 dishes with serving size and nutrition per 100 g and per serving (for 'fix the dish' or 'add a dish' screens)
GET /health	status, model version, CPU threads (no key needed)
GET /docs	interactive documentation of every field

```
curl -H "X-API-Key: <key>" -F "image=@meal.jpg" https://<server>/v1/analyze
```

```
{"items": [{"dish": "roti", "name": "Roti / chapati", "confidence": 0.996, "unsure": false,
            "alternatives": [{"name": "Plain paratha", "confidence": 0.002}, ...],
            "count": 1.0, "unit": "1 medium roti", "grams": 40.0,
            "kcal": 80.9, "protein_g": 2.4, "carb_g": 14.3, "fat_g": 1.4, "fibre_g": 2.5,
            "boxes": [[0.3519, 0.536, 0.7235, 0.9991]]}, ...],
 "totals": {"kcal": 416.9, "protein_g": 16.1, "carb_g": 54.5, "fat_g": 15.2, "fibre_g": 12.2, "grams": 390.0},
 "image_width": 1200, "image_height": 1600, "processing_ms": 1044, "model_version": "76ec58bc09aa"}
```

- **unsure** is true when confidence is below 0.4: show the alternatives or ask the user to confirm the dish.
- **count** is the number of pieces for countable dishes (roti, idli, egg) and the number of servings for everything else. Let users edit it: the nutrition scales linearly.

- **carb_g** is available carbohydrate (fibre not included). **boxes** are [x1, y1, x2, y2] fractions of the photo, so the app can draw them.
- **Errors** return JSON {"detail": ...}: 400 not an image, 401 wrong or missing key, 413 photo too large, 422 'image' field missing.

9. Updating the models or nutrition values

- To replace the models or the nutrition table, swap the files in models/ or data/nutrition/dish_nutrition.csv, rebuild the image and restart. model_version in /health changes automatically.
- Retraining (new dishes, more photos) is done in the project repository on a free Kaggle GPU; it produces a new food-plate-api.zip with python -m plate.build_deploy.

10. Known limitations

- **Portions** are standard servings (1 katori, 1 roti), not measured from the photo.
- **Gravy over rice** (rajma-chawal, dal-chawal on one plate) is often read as plain rice.
- **Touching pieces** (a pile of momos, overlapping rotis) may be counted as 1.
- **About 25% of found dishes get the wrong name**, usually a similar dish such as mixed veg vs. dry sabzi.
- **Only 110 dishes are recognised.** Other dishes get the closest known name. Expanding the dish list is the next phase.
- The training and test labels were produced with AI assistance and have **not yet been checked by a person**.

11. Licences: decide before a public launch

> **Ultralytics YOLO is AGPL-3.0.** The detector library, and the YOLO11 weights it was fine-tuned from, are AGPL-3.0. Internal testing is fine. A service offered to outside users must either make its source code available to them or buy an Ultralytics Enterprise licence. The third option is to retrain the detector on a permissively licensed model, for example RT-DETR in Hugging Face transformers (Apache-2.0); that means retraining and re-testing.

Component	Licence / note
SigLIP base model, transformers, PyTorch, FastAPI, uvicorn	Apache-2.0 / BSD / MIT: fine for commercial use
Training photos	company user photos plus 6 public food datasets. Some of the public datasets don't state a licence; check them before commercial use (list in the report).
Nutrition values	INDB, IFCT 2017 (NIN-ICMR), USDA FoodData Central (public domain). The source of each dish's values is in dish_nutrition.csv. Check the INDB and IFCT terms before showing the values in a commercial product.

12. Troubleshooting

Symptom	Fix
libGL.so.1 error (bare Linux)	`sudo apt install libgl1 libglib2.0-0`
More than 3 s per photo	check cpu_threads in /health; give the container more CPUs and set PLATE_TORCH_THREADS to match
Container killed / out of memory	allow about 1.2 GB per worker; fewer workers or more RAM

Symptom	Fix
413 on large photos	raise PLATE_MAX_UPLOAD_MB and the proxy's body-size limit, or resize on the phone (about 1600 px on the long side is plenty)
400 for iPhone HEIC files	send JPEG; browsers and most upload libraries convert HEIC automatically. Or install pi-heif==1.4.0, which is optional and not tested yet.
Health check fails right after start	the models are still loading; allow 60–90 s
'... is a Git LFS pointer' at start	the repository was cloned without git-lfs: run *git lfs install* and *git lfs pull*, then rebuild
docker build fails	send the last 30 lines of the build output